

Linq und Erweiterungsmethoden

Linq

- Language **I**ntegrated **Q**uery
- Collections verarbeiten
 - alle Klassen die von IEnumerable erben (List, Array, Dictionary, ...)
- SQL-ähnliche Verwendung (filtern, sortieren, summieren, ...)
- Lambda-Expressions (=>)
- verketteten von einzelnen Linq-Abfragen
- verändern **nicht** die originale Collection
 - Zuweisung auf die originale Collection um Änderungen zu „Übernehmen“

Aufbau

```
List<int> list = new List<int> { 1, 2, 3, 4, 5, 6, 7, 8, 9 };  
    list.Where(e => e % 2 == 0).ToList();
```

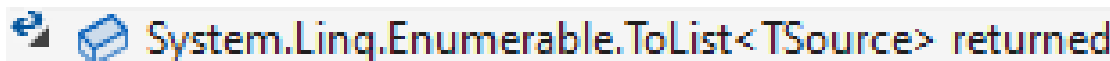
Originale
Collection

Linq-
Funktion

Lambda-Expression

Linq-
Funktion

- Finde alle geraden Zahlen und erzeuge eine neue Liste
- mit Variablennamen (e) und => eine Bedingung festlegen
- abschließend mit ToList() das aus Where(...) resultierende IEnumerable<int> zu einer List<int> konvertieren

	Count = 4	System.Collections.Generic.List<int>
 [0]	2	int
 [1]	4	int
 [2]	6	int
 [3]	8	int

Einfache Linq-Funktionen

```
List<int> list = new List<int> { 1, 2, 3, 4, 5, 6, 7, 8, 9 };
```

Funktionsname	Beschreibung	Ergebnis
Average	Gibt den Durchschnitt der Liste zurück	5
Min/Max	Gibt den kleinsten/größten Wert der Liste zurück	1, 9
Sum	Gibt die Summe der Liste zurück	45
First	Gibt das erste Element der Liste zurück, wirft eine Exception wenn kein Element gefunden wird	1
FirstOrDefault	Erstes Element, null wenn kein Element gefunden wird	1
Last	Gibt das letzte Element der Liste zurück, wirft eine Exception wenn kein Element gefunden wird	9
LastOrDefault	Letztes Element, null wenn kein Element gefunden wird	9

Vergleich Linq-Schreibweisen

```
List<int> list = new List<int>() { 1, 2, 3, 4, 5, 6, 7, 8, 9 };
```

```
List<int> geradeForEach = new List<int>();  
foreach (int i in list)  
    if (i % 2 == 0)  
        geradeForEach.Add(i);
```

```
List<int> geradeSQL = (from i in list  
                      where i % 2 == 0  
                      select i).ToList();
```

```
List<int> geradeLinq = list.Where(e => e % 2 == 0).ToList();
```

Linq mit Objektliste

```
List<Fahrzeug> fahrzeuge = new List<Fahrzeug>()...
```

Funktionsname	Beschreibung	Ergebnis
Where	Filtert die Liste nach einem oder mehreren Kriterien	IEnumerable<Fahrzeug>
Select	Projiziert die Liste auf eine neue Liste nach einem Kriterium	IEnumerable<?>
OrderBy und OrderByDescending	Gibt eine neue Liste sortiert (auf- oder absteigend) nach einem Kriterium zurück	IEnumerable<Fahrzeug>
All/Any	Überprüft ob alle/mindestens ein Element in die Liste die angegebene Bedingung erfüllt	bool
Count/LongCount	Zähle die Elemente in der Liste anhand eines/mehreren Kriterien	int/long
MinBy/MaxBy	Gibt das Objekt mit dem kleinsten/größten Wert anhand eines Kriteriums zurück	Fahrzeug
Skip/Take	Überspringt X Elemente, nimmt die nächsten X Elemente. In Kombination verwenden um Teile der Liste zu nehmen	

Linq mit Objektliste (2)

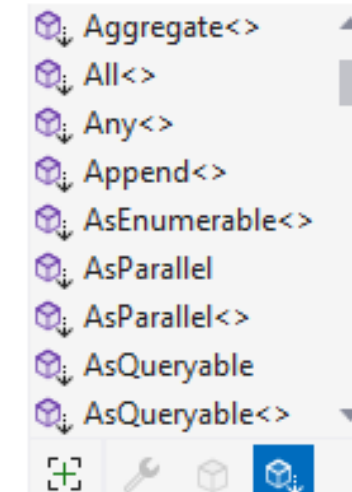
Funktionsname	Beschreibung	Ergebnis
SequenceEqual	Überprüft, ob zwei Listen die selben Elemente haben	bool
Reverse	Erzeugt eine neue Liste die die originale Liste nur umgekehrt ist	IEnumerable<Fahrzeug>
Concat	Verbindet zwei Listen zu einer neuen Liste	IEnumerable<Fahrzeug>
Prepend/Append	Erzeugt eine neue Liste und fügt das Element am Anfang/Ende an	IEnumerable<Fahrzeug>
Except	Gibt eine neue Liste zurück mit Elementen die nur in einer der beiden Listen ist	IEnumerable<Fahrzeug>
Intersect	Gibt die Schnittmenge zwischen den beiden Listen zurück	IEnumerable<Fahrzeug>
SelectMany	Glättet eine Liste von Listen auf eine einzelne Liste herunter	IEnumerable<Fahrzeug>
Chunk	Teilt die Liste auf Teile mit der angegebenen Größe auf	IEnumerable<Fahrzeug[]>
GroupBy	Gruppiert die Liste nach einem Kriterium	IEnumerable<IGrouping<?, Fahrzeug>>
Aggregate	Wendet für jedes Element eine Funktion auf einen Aggregator an	? (Typ des Aggregators)

Konvertierungsfunktionen

- Originale Collection in eine andere Form umwandeln
 - List ↔ Array
 - IEnumerable → List
 - IEnumerable<IGrouping<?, Fahrzeug>> (GroupBy) → Dictionary
- ToArray
- ToDictionary
- ToHashSet
- ToList
- AsEnumerable

Erweiterungsmethoden

- alle Linq Methoden sind Erweiterungsmethoden
- beliebige Methoden an Typen anhängen
- gekennzeichnet mit Würfel + Pfeil nach unten oder mit (extension) bei Methodenvorschau
- Aufbau
 - müssen in einer statischen Klasse definiert werden
 - müssen selbst auch statisch sein
 - mit `this <Typ>` sich auf einen Typen beziehen



Show only Extension methods (Alt+X)

(extension) IEnumerable<int>
Filters a sequence of values ba

Eigene Erweiterungsmethode

```
public static int GetQuersumme(this int zahl)
{
    int summe = 0;
    string zahlAlsString = zahl.ToString();
    for(int i=0;i<zahlAlsString.Length; i++)
    {
        summe += (int) char.GetNumericValue(zahlAlsString[i]);
    }
    return summe;
}
```

This zeigt an für welche Klasse eine Erweiterungsmethode definiert werden soll

```
Console.WriteLine(10232.GetQuersumme()); //Gibt 8 zurück
```